

INFORME IMPLEMENTACIÓN Y COMPARACIÓN DE ALGORITMOS DE BÚSQUEDA

Nombre: José Villablanca Alarcón

1. Introducción y Objetivos

Este documento presenta de forma ejecutiva la implementación y el análisis comparativo de cuatro algoritmos fundamentales de búsqueda en espacios de estados: Búsqueda en Anchura (BFS), Búsqueda en Profundidad (DFS), Búsqueda de Costo Uniforme (UCS) y Búsqueda A*. El objetivo principal es evaluar su eficiencia temporal (tiempo de ejecución), complejidad espacial (nodos visitados) y optimalidad bajo dos entornos de prueba:

- **Grafos Orientados a Conectividad (Redes de Ciudades y Árboles Binarios):** Escenarios con topologías explícitas y pesos variables en las aristas para analizar el impacto de los costos.
- **Laberintos Matriciales:** Entornos bidimensionales basados en rejillas con obstáculos y costos de transición unitarios, ideales para contrastar la búsqueda informada y no informada.

2. Decisiones de Diseño

- **Arquitectura Desacoplada:** Se implementó una interfaz implícita mediante un Protocolo (typing.Protocol) en Python denominado Escenario. Esto permite que los algoritmos interactúen de forma idéntica con grafos y laberintos sin requerir detalles internos de su almacenamiento.
- **Estructuras de Datos Clave:**
 - **BFS:** Utiliza una cola FIFO basada en collections.deque para garantizar inserciones y extracciones en tiempo constante $O(1)$.
 - **DFS:** Emplea una pila LIFO mediante listas nativas de Python, operando con append() y pop() en tiempo amortizado $O(1)$.
 - **UCS y A*:** Utilizan colas de prioridad mediante el módulo heapq ($O(\log N)$). Para prevenir errores de tipo (TypeError) al desempatar nodos con la misma prioridad, los elementos del montículo se estructuraron como tuplas del tipo (costo_f, contador, nodo), donde el contador es un entero estrictamente monótono.
- **Admisibilidad Matemática de la Heurística:** En los laberintos matriciales se aplicó la Distancia de Manhattan como función heurística. Es matemáticamente admisible ($0 \leq h(n) \leq h^*(n)$) porque el movimiento está restringido a las cuatro direcciones ortogonales con costo 1.0. En un escenario ideal libre coincide exactamente con la distancia mínima real, mientras que la presencia de muros solo puede obligar a

desvíos que incrementen el costo real ($h^*(n) \geq h(n)$), asegurando la optimalidad de A*.

3. Resultados y Análisis Crítico

Ciudades

NODOS DISPONIBLES: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
→ Nodo inicial: A
→ Nodo objetivo: F

👉 Inicio: A | Objetivo: F

Ejecutando algoritmos...

TABLA COMPARATIVA DE RESULTADOS

Algoritmo	Costo	Nodos visitados	Long. camino	Óptimo	Tiempo
BFS	25	6	4	✗ No	0.061 ms
DFS	11	4	4	✓ Sí	0.107 ms
UCS	11	6	4	✓ Sí	0.320 ms
A*	11	6	4	✓ Sí	0.521 ms

✓ Arbol binario cargado. Nodos: [1, 2, 3, 4, 5, 6, 7]

NODOS DISPONIBLES: [1, 2, 3, 4, 5, 6, 7]
→ Nodo inicial: 1
→ Nodo objetivo: 6

👉 Inicio: 1 | Objetivo: 6

Ejecutando algoritmos...

TABLA COMPARATIVA DE RESULTADOS

Algoritmo	Costo	Nodos visitados	Long. camino	Óptimo	Tiempo
BFS	2	6	3	✓ Sí	0.069 ms
DFS	2	4	3	✓ Sí	0.016 ms
UCS	2	6	3	✓ Sí	0.032 ms
A*	2	6	3	✓ Sí	0.024 ms

```

DIMENSIONES DEL LABERINTO: 7 filas x 7 columnas
Representación textual del laberinto:

S . | . . .
| . | . .
| | . | .
| | | | .
| | | | | G
. . . . .

Inicio actual: (0, 0). ¿Usar este? [S/n]: S
Objetivo actual: (6, 6). ¿Usar este? [S/n]: S

‣ Inicio: (0, 0) | Objetivo: (6, 6)

Ejecutando algoritmos...

```

TABLA COMPARATIVA DE RESULTADOS

Algoritmo	Costo	Nodos visitados	Long. camino	Óptimo	Tiempo
BFS	12	27	13	✓ Sí	0.492 ms
DFS	12	13	13	✓ Sí	0.354 ms
UCS	12	27	13	✓ Sí	0.375 ms
A*	12	13	13	✓ Sí	0.171 ms

```

DIMENSIONES DEL LABERINTO: 10 filas x 10 columnas
Representación textual del laberinto:

S . . . . .
| . | . | . | .
| | . | . | .
| | | | . | .
| | | | | .
| | | | | | G
| | . . . . .

Inicio actual: (0, 0). ¿Usar este? [S/n]: s
Objetivo actual: (9, 9). ¿Usar este? [S/n]: s

‣ Inicio: (0, 0) | Objetivo: (9, 9)

Ejecutando algoritmos...

```

TABLA COMPARATIVA DE RESULTADOS

Algoritmo	Costo	Nodos visitados	Long. camino	Óptimo	Tiempo
BFS	18	44	19	✓ Sí	0.258 ms
DFS	18	38	19	✓ Sí	0.155 ms
UCS	18	44	19	✓ Sí	0.205 ms
A*	18	34	19	✓ Sí	0.186 ms

A partir de las métricas exactas recolectadas en los escenarios de prueba estándar (Grafo de Ciudades, Árbol Binario, Laberinto 7x7 y Laberinto 10x10), se derivan las siguientes conclusiones analíticas:

- DFS (No-optimalidad y Exploración Inestable):** No garantiza encontrar el camino más corto en costo ni en aristas. Aunque en entornos geométricos avanzó de forma muy agresiva visitando solo 4 nodos en el Árbol Binario, 13 en el Laberinto 7x7 y 38 en el 10x10, su estrategia es arriesgada. Su obtención constante del costo mínimo en todas las pruebas (11 en ciudades, 2 en árbol, 12 y 18 en laberintos) se debió a

una coincidencia fortuita por el orden de inserción de nodos. Si entra en callejones sin salida profundos, el backtracking exhaustivo degrada drásticamente su eficiencia al carecer de una evaluación real de costos.

- **BFS (Optimalidad Estructural y Limitaciones con Pesos Variables):** Garantiza la ruta con la menor cantidad de pasos o saltos. Logró las longitudes óptimas de 3 pasos en el Árbol Binario y de 13 y 19 pasos en los laberintos 7x7 y 10x10. Sin embargo, en el Grafo de Ciudades demostró su ineficacia ante pesos variables: al asumir aristas uniformes, prefirió un camino corto en saltos pero con un costo elevado de 25, ignorando la ruta óptima de 11. Además, su exploración radial genera una alta redundancia espacial, obligándolo a expandir muchos estados (27 y 44 nodos en los laberintos).
- **UCS (Robustez Óptima frente a Carga Computacional):** Garantiza siempre el camino de menor costo absoluto en cualquier escenario, registrando con éxito los mínimos reales: 11 en ciudades, 2 en árbol, 12 en el laberinto 7x7 y 18 en el 10x10. Su desventaja es que realiza una búsqueda ciega y radial. Al no tener dirección, en escenarios con pesos homogéneos replica exactamente la cantidad de nodos visitados de BFS (6 en árbol, 27 en laberinto 7x7 y 44 en 10x10), pero arrastrando una mayor sobrecarga de tiempo (0.320 ms en ciudades) por la gestión interna de la cola de prioridad.
- **A* (Eficiencia Directiva mediante Heurística):** Combina la exactitud de UCS con el enfoque dirigido de una función heurística $f(n) = g(n) + h(n)$. Destaca notablemente en problemas espaciales: en el Laberinto 7x7 redujó la búsqueda a solo 13 nodos (un 51.8% menos que UCS/BFS) en 0.171 ms. En el Laberinto 10x10 limitó la exploración a 34 nodos (un ahorro del 22.7%) en 0.186 ms. Cuando la heurística no aporta ventaja espacial (como los 6 nodos del Árbol Binario), se comporta de forma idéntica a UCS, pero consolidándose como el algoritmo más eficiente y equilibrado del estudio.

4. Conclusiones y Recomendaciones de Selección

El análisis empírico fundamenta los siguientes criterios de ingeniería de software para la selección de algoritmos de búsqueda:

1. **Escenarios de Pesos Uniformes (Sin heurística):** La mejor opción es BFS. Ofrece optimalidad en número de pasos y una alta eficiencia al prescindir de la complejidad de ordenación de una cola de prioridad.
2. **Escenarios de Pesos Variables (Sin heurística viable):** El uso de UCS es indispensable. Es el único mecanismo no informado que asegura la ruta de costo acumulado mínimo, asumiendo una exploración exhaustiva multidireccional.
3. **Escenarios con Información Espacial / Heurística Admisible:** La solución superior por excelencia es A*. Al integrar métricas admisibles como la distancia de Manhattan, minimiza significativamente el uso de memoria (nodos visitados) y optimiza el tiempo de procesamiento, garantizando siempre la ruta óptima absoluta.

4. **Búsqueda Rápida en Grandes Espacios (Sin requerir optimalidad):** Se puede utilizar DFS con cautela. Es de utilidad si los recursos de memoria del sistema son severamente limitados o si solo se necesita verificar la mera existencia de un camino, aceptando el riesgo de obtener rutas altamente ineficientes.